

Technische Hochschule Deggendorf

Case study project -
Autonomous LLM-Based Computer-Use Agents for End-to-End Testing of Production Software Systems

Industrial Application: MES – Spare-Parts & Maintenance Management

Bibliography Report

Submitted by:

Group 5

Asem Marwad

Ibram Gergis

Course:

Case Study: Intelligent Systems

Under the Supervision of:

Prof. Dr. Hamidreza Heidari

Date of Submission: 02.11.2026

Contents

Abstract

1 Introduction

1.1 Objectives

1.2 Scope of Study

2 Bibliography Summary and Literature Alignment

2.1 Overview

2.2 Summary of Reviewed Research Works

2.2.1 Software Testing with Large Language Models: Survey, Landscape, and Vision

2.2.2 Why Do Record/Replay Tests of Web Applications Break?

2.2.3 ReAct: Synergizing Reasoning and Acting in Language Models

2.2.4 Mind2Web: Towards a Generalist Agent for the Web

2.2.5 WebArena: A Realistic Web Environment for Building Autonomous Agents

2.2.6 GPT-4(vision) is a Generalist Web Agent, if Grounded (SeeAct)

2.2.7 OSWorld: Benchmarking Multimodal Agents in Real Computer Environments

2.2.8 GPTDroid: Bringing Human-like Interaction to Mobile GUI Testing

2.2.9 DroidAgent: Intent-Driven Mobile GUI Testing with Autonomous LLM Agents

2.2.10 The Oracle Problem in Software Testing: A Survey

2.3 Summary of Reviewed Works

2.4 Synthesis and Observations

2.5 Alignment with the Proposed Work

2.5.1 Identified Research Gaps

3 Approach Selection Justification

3.1 Reasons for Selection

3.2 Technical Stack and Benchmark Setup

3.3 Conclusion

References

List of Tables

2.1 Reviewed approaches to automated GUI and end-to-end testing with their reported results.

2.2 Alignment of reviewed research insights with the proposed testing approach.

Abstract

Modern manufacturing under the paradigm of **Industry 4.0** depends on software as much as on machines. **Manufacturing Execution Systems (MES)** coordinate work orders, equipment maintenance and spare-parts inventory, and a single defect in these workflows, such as a material issue that drives stock negative or a low-stock alert that never fires, can stall a repair and extend machine downtime. Today these workflows are verified either manually, which is slow and inconsistent, or through **scripted end-to-end tests** that must be written and maintained by engineers for every scenario and that break whenever the user interface changes.

This report presents a bibliographic study of recent research on **large language model (LLM) agents** that operate graphical user interfaces and on their use in software testing. It reviews work on LLM-based software testing, the fragility of scripted web tests, reasoning-and-acting agent architectures, web and desktop agent benchmarks (Mind2Web, WebArena, SeeAct, OSWorld) and LLM-driven GUI testing tools (GPTDroid, DroidAgent), together with the test-oracle problem. The analysis compares these approaches in terms of task success, robustness to interface change, verification reliability and suitability for industrial applications.

By synthesizing these insights, the report establishes the foundation for an **autonomous computer-use testing agent** that converts natural-language test instructions into GUI actions on an MES, checks the outcome with a **deterministic verification engine** based on read-only database and API checks, and is benchmarked against a conventional **Playwright** test suite on spare-parts and maintenance scenarios.

1. Introduction

Manufacturing Execution Systems sit between enterprise planning and the shop floor, managing production orders, maintenance and material flows [1, 2]. As factories move towards connected, data-driven operation under Industry 4.0 [3], the correctness of this software becomes a direct operational concern: when a technician requests a replacement bearing for a CNC machine, the system must confirm that the part exists, reserve or

issue the correct quantity, update the stock balance, raise a reorder warning where needed and record the transaction against the right work order. Testing such workflows by hand is time-consuming, while scripted automation requires a dedicated test script for every scenario. This chapter introduces the motivation for using LLM-based computer-use agents as an adaptive testing layer that understands test instructions written in natural language and executes them through the user interface in the way a human tester would.

1.1 Objectives

- To review existing literature on LLM-based software testing and autonomous GUI and computer-use agents.
- To identify research gaps in applying these agents to end-to-end testing of industrial applications such as MES.
- To justify the use of an LLM computer-use agent combined with a deterministic verification engine, benchmarked against conventional scripted Playwright testing, for spare-parts and maintenance workflows.

1.2 Scope of Study

This review covers publications and frameworks from 2015–2025, with an emphasis on 2023–2025, focusing on LLM-based testing, web and desktop GUI agents, scripted end-to-end testing and test oracles. Industrial MES literature is included to frame the application domain. Demand forecasting, inventory optimization and control of physical machines are outside the scope.

2. Bibliography Summary and Literature Alignment

2.1 Overview

This section summarizes the key research studies that influenced the design of the **autonomous MES testing agent**. The reviewed works trace the development from scripted end-to-end testing towards **LLM agents that observe and operate real interfaces**, focusing on reasoning-and-acting loops, visual grounding, realistic benchmarks, LLM-driven GUI testing and the problem of deciding automatically whether an observed behaviour is correct.

2.2 Summary of Reviewed Research Works

2.2.1 Software Testing with Large Language Models: Survey, Landscape, and Vision

Authors: Junjie Wang et al. (2024), *IEEE Transactions on Software Engineering*[4]

This survey analyses 102 studies that apply large language models to software testing and organizes them by testing task (unit test generation, test oracle generation, input generation, debugging and repair) and by how the LLM is used (prompt engineering, fine-tuning and combination with traditional techniques). The authors find that most work concentrates on unit-level code testing, while system-level and GUI-level testing receive far less attention. They identify open challenges in reaching high coverage, in reliable test oracles and in rigorous evaluation, and call for applying LLMs to more realistic, domain-specific systems.

Key Insight: LLMs are a promising testing tool, but system-level GUI testing and the oracle problem remain under-explored.

Relevance: Our project targets exactly this gap: system-level, end-to-end testing of an industrial application through its GUI.

2.2.2 Why Do Record/Replay Tests of Web Applications Break?

Authors: Mouna Hammoudi, Gregg Rothermel and Paolo Tonella (2016), *IEEE ICST*[5]

This empirical study examines why recorded and replayed end-to-end tests of web applications stop working as the applications evolve. Analysing test breakages across successive releases of several open-source web applications, the authors build a taxonomy of breakage causes and find that changes to element **locators** (identifiers, XPath and CSS paths) are the most frequent cause, followed by changes in values, page reloads and pop-up boxes. The study complements broader reviews of end-to-end web testing approaches and tools [6] and later work on repairing broken tests with visual information [7].

Key Insight: Scripted GUI tests are deterministic but brittle; much of their maintenance cost comes from locators that no longer match the interface.

Relevance: An agent that finds elements from their visible meaning rather than fixed selectors may reduce this maintenance; this is measured against a Playwright baseline [8] in our evaluation.

2.2.3 ReAct: Synergizing Reasoning and Acting in Language Models

Authors: Shunyu Yao et al. (2023), *ICLR*[9]

ReAct proposes that an LLM should interleave **reasoning traces** with **actions** in an environment, so that each observation updates its plan before the next action. On the interactive benchmarks ALFWorld and WebShop,

ReAct outperforms imitation and reinforcement learning baselines by 34% and 10% absolute success rate respectively, while producing interpretable decision traces. Follow-up work such as Reflexion [10] adds verbal self-reflection after failed attempts, and surveys of LLM-based autonomous agents [11] identify this plan–act–observe loop as the common core of agent architectures.

Key Insight: Interleaving reasoning with observation lets an agent adapt to unexpected states instead of following a fixed script.

Relevance: Our computer-use agent follows a ReAct-style loop with a bounded number of retries, and its reasoning trace becomes part of the test evidence.

2.2.4 Mind2Web: Towards a Generalist Agent for the Web

Authors: Xiang Deng et al. (2023), *NeurIPS*[12]

Mind2Web introduces the first large dataset for generalist web agents, containing over 2,000 open-ended tasks collected from 137 real websites in 31 domains, each with crowd-sourced action sequences. The authors propose MindAct, which first filters the large HTML page with a small ranking model and then lets an LLM select the element and operation. The results show that LLMs can generalize to unseen websites to a useful degree, but complete-task success remains low, because a single wrong step in a long sequence causes the whole task to fail.

Key Insight: Real, cluttered web pages are hard for agents, and errors accumulate over long action sequences.

Relevance: MES workflows are multi-step, so our agent breaks each test into short verified steps and records every action for reproduction.

2.2.5 WebArena: A Realistic Web Environment for Building Autonomous Agents

Authors: Shuyan Zhou et al. (2024), *ICLR*[13]

WebArena provides a **self-hosted, reproducible** web environment with fully functional applications for e-commerce, forums, software development and content management, together with 812 long-horizon tasks. Task success is checked through **programmatic validation of the resulting application state** rather than by comparing action sequences. The best GPT-4-based agent completed only 14.41% of the tasks, compared with 78.24% for human participants, showing a large gap between current agents and people on realistic tasks.

Key Insight: Reproducible self-hosted applications and state-based checks make agent evaluation reliable.

Relevance: We adopt the same principle: a locally deployed MES with synthetic data, and verification through read-only database and API checks.

2.2.6 GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)

Authors: Boyuan Zheng et al. (2024), *ICML*[14]

SeeAct studies large multimodal models that act on websites from screenshots. When its textual action plans are grounded into concrete page elements by a human, GPT-4V can complete 51.1% of tasks on live websites, substantially more than text-only models. However, automatic **grounding**, the mapping from an intended action to the correct element on screen, remains the main bottleneck; combining HTML structure with visual information works best. WebVoyager [15] reports a similar result for an end-to-end multimodal agent, which completed 59.1% of tasks on 15 popular real-world websites.

Key Insight: Screenshot-based agents can plan well, but reliable grounding requires combining vision with page structure.

Relevance: Our agent uses screenshots together with the browser accessibility tree, as offered by current computer-use tools [16, 17].

2.2.7 OSWorld: Benchmarking Multimodal Agents in Real Computer Environments

Authors: Tianbao Xie et al. (2024), *NeurIPS Datasets and Benchmarks*[18]

OSWorld is a benchmark of 369 tasks executed in real desktop operating systems and applications, with an execution-based evaluation script for every task. Humans complete 72.36% of the tasks, whereas the best model evaluated at publication completed only 12.24%. The authors attribute the gap mainly to weak GUI grounding and limited operational knowledge, and show that performance drops further with high-resolution screens and long interaction histories.

Key Insight: General computer-use agents are still unreliable on open-ended tasks, even though their capabilities are improving quickly.

Relevance: Our project uses a narrow, well-specified domain and explicit verification so that agent errors are detected and reported as AGENT ERROR rather than mistaken for software defects.

2.2.8 GPTDroid: Bringing Human-like Interaction to Mobile GUI Testing

Authors: Zhe Liu et al. (2024), *ICSE*[19]

GPTDroid formulates mobile GUI testing as a question-and-answer dialogue: the current GUI page is described to the LLM, which decides the next action, and a functionality-aware memory keeps track of which app features have been tested. On a benchmark of popular Android apps, the approach achieves 32% higher activity coverage than the best baseline and detects 31% more bugs at a faster rate. It also found 53 new bugs in Google Play apps, 35 of which were confirmed and fixed by developers.

Key Insight: LLMs that reason about application functionality explore and test GUIs better than random or model-based testing tools.

Relevance: Our agent plans around business functions (issue, reserve, low-stock check) rather than around individual screens.

2.2.9 DroidAgent: Intent-Driven Mobile GUI Testing with Autonomous LLM Agents

Authors: Juyeon Yoon, Robert Feldt and Shin Yoo (2024), *IEEE ICST*[20]

DroidAgent is an autonomous LLM agent for Android GUI testing that sets its own realistic testing goals, or intents, and pursues them with planning, acting and reflection components supported by long-term memory. In the evaluation it reached 61% activity coverage, compared with 51% for state-of-the-art GUI testing tools, and manual analysis found that 317 of the 374 tasks it created were realistic and relevant to the application's functions.

Key Insight: Intent-driven agents produce meaningful, human-like test scenarios rather than random input sequences.

Relevance: Our system takes the intent directly from natural-language test instructions written for MES spare-parts and maintenance scenarios.

2.2.10 The Oracle Problem in Software Testing: A Survey

Authors: Earl T. Barr et al. (2015), *IEEE Transactions on Software Engineering*[21]

This survey defines the **test oracle problem**: deciding whether the behaviour observed for a given input is correct. It reviews specified oracles (formal specifications and assertions), derived oracles (regression, metamorphic and pseudo-oracles) and implicit oracles (crashes and exceptions), and argues that the human effort needed to judge outputs is a major limit on test automation. This problem is especially relevant for LLM agents, since LLM judgements are known to show position, verbosity and self-enhancement biases [22].

Key Insight: Automated testing is only as reliable as its oracle; a judgement from an LLM alone is not a sufficient oracle.

Relevance: Our verification engine uses specified oracles derived from business rules, such as “stock must never become negative”, and checks them against the database and API state.

2.3 Summary of Reviewed Works

Approach / Work	Observation and interaction	Reported result	Remarks
Scripted E2E testing (Selenium / Playwright) [5, 6, 8]	DOM selectors, scripted steps	Deterministic; locator changes are the most frequent cause of breakage	Fast and repeatable, but high authoring and maintenance effort for every scenario.
ReAct [9]	Text observations, tool actions	+34% / +10% absolute success (ALFWorld / WebShop)	Reason–act–observe loop; basis for adaptive agents.
MindAct / Mind2Web [12]	Filtered HTML, element selection	Low complete-task success on unseen websites	Large real-world dataset; errors accumulate over long tasks.
WebArena GPT-4 agent [13]	Accessibility tree, browser actions	14.41% task success (humans 78.24%)	Self-hosted applications with state-based evaluation.
SeeAct [14]	Screenshots + HTML	51.1% task success with oracle grounding	Grounding is the main bottleneck.
WebVoyager [15]	Screenshots, browser actions	59.1% task success on 15 real websites	End-to-end multimodal web agent.
OSWorld best model [18]	Desktop screenshots, mouse and keyboard	12.24% task success (humans 72.36%)	Execution-based checks on real operating systems.
GPTDroid [19]	GUI page text, LLM dialogue	+32% activity coverage, +31% bugs detected vs. best baseline	Functionality-aware LLM testing; 53 new bugs found.
DroidAgent [20]	GUI state, planner +	61% activity coverage	Intent-driven, human-like test

	reflector	(baseline 51%)	scenarios.

Table 2.1: Reviewed approaches to automated GUI and end-to-end testing with their reported results.

2.4 Synthesis and Observations

The reviewed literature shows a clear development from scripted, selector-based tests towards LLM agents that reason about the task and observe the interface directly. The key patterns observed are:

- Scripted tests are precise and cheap to run, but their locators break when the interface changes, and each scenario must be written by hand.
- Reasoning-and-acting loops let agents recover from unexpected states, but long action sequences multiply the chance of a single fatal error.
- Visual grounding is the main weakness of screenshot-based agents; combining screenshots with page structure gives the best results.
- Benchmarks with self-hosted applications and state-based checks (WebArena, OSWorld) give the most trustworthy measurements.
- LLM-driven testing tools find real bugs in mobile apps, but almost no work evaluates them on industrial applications or business-rule defects.
- An LLM's judgement of a screenshot is not a reliable oracle on its own; explicit, rule-based verification is needed.

2.5 Alignment with the Proposed Work

Criteria	**Research Insight**	**Proposed Approach Justification**
Robustness to UI change	Locator changes are the main cause of scripted-test breakage	The agent finds elements by their visible meaning, so layout changes do not require rewriting tests

Test authoring effort	Every scripted scenario needs hand-written code	Tests are written as natural-language instructions with expected business rules
Grounding accuracy	Screenshot-only agents mis-select elements	The agent combines screenshots with the accessibility tree of the MES pages
Verification reliability	LLM judgements are biased; the oracle problem limits automation	A deterministic verification engine checks database and API state against business rules
Handling of unexpected states	Agents fail on long sequences and pop-ups	ReAct-style observe–reassess loop with a bounded number of retries
Industrial relevance	Benchmarks use generic web and desktop applications	Evaluation on MES spare-parts and maintenance workflows with injected, known defects

Table 2.2: Alignment of reviewed research insights with the proposed testing approach.

2.5.1 Identified Research Gaps

- Lack of studies applying LLM computer-use agents to end-to-end testing of industrial applications such as MES.
- Minimal separation between the agent that acts and the oracle that decides the verdict; many systems let the LLM judge its own result.
- Few controlled comparisons of LLM agents against conventional scripted testing using known, injected defects.
- Limited reporting of cost, execution time and human intervention alongside detection accuracy.

3. Approach Selection Justification

Based on the literature analysis, an **LLM-based computer-use agent** combined with a **deterministic verification engine** was selected as the core approach for this project, with a conventional **Playwright** test suite as the baseline. The agent provides natural-language test authoring and adaptation to interface changes, while the verification engine provides the reliable oracle that the agent alone cannot. The application under test is a controlled, locally deployed MES, with OpenMES [23] as the candidate, populated with synthetic spare-parts and work-order data.

3.1 Reasons for Selection

- **LLM computer-use agent:** interprets natural-language test instructions, plans the workflow, and operates the MES through screenshots, mouse and keyboard in a ReAct-style loop with bounded retries.
- **Deterministic verification engine:** checks business rules such as non-negative stock, correct stock deduction, reorder thresholds and no duplicate transactions through UI assertions and read-only database and API checks, which addresses the oracle problem.
- **Playwright baseline:** provides the conventional scripted reference against which detection recall, precision, false-positive rate, execution time, cost per test and human intervention are compared.
- **Combined Advantage:** separates acting from judging, so that every test ends in a clear PASS, FAIL, INCONCLUSIVE or AGENT ERROR verdict supported by screenshots, action logs and reproduction steps.

3.2 Technical Stack and Benchmark Setup

The prototype is built from widely used components so that every experiment can be repeated on a single workstation.

- **Language model:** GLM-5.3 Flash plans each test step and chooses the next GUI action from the observed page state.
- **Agent and browser control:** Python 3.11 with Playwright executes the agent's clicks and keyboard input and captures a screenshot and the accessibility tree after every step.
- **System under test:** OpenMES (Laravel and PostgreSQL), deployed locally with Docker and populated with synthetic spare-parts and work-order data.
- **Verification engine:** read-only queries against the OpenMES PostgreSQL database and REST API evaluate the business rules defined for each scenario.
- **Test management:** scenarios and injected defects are described in YAML files, executed with pytest

and run on every change through GitHub Actions.

The benchmark compares three approaches: the Playwright baseline, the agent without verification (pure agent) and the agent with the verification engine. Each runs three scenarios, insufficient stock (SP-01), low-stock warning (SP-02) and stock balance update (SP-03), and each scenario contains one injected defect (DEF-01 to DEF-03). The approaches are compared on detection recall, precision, false-positive rate, execution time, cost per test and the number of human interventions.

3.3 Conclusion

The reviewed literature shows that LLM agents can now operate graphical user interfaces, but they have not yet been used for the systematic testing of industrial software. Scripted tests are precise and repeatable, yet costly to write and fragile when the interface changes. Computer-use agents adapt to the interface, but they still make grounding errors and cannot be trusted to judge their own results.

This project therefore separates acting from judging: the agent acts, and fixed business rules judge the result. An LLM computer-use agent carries out each test through the MES user interface, and a deterministic verification engine decides the verdict from the database and API state. This addresses the research gaps identified in Section 2.5.1:

- **Industrial evaluation:** the approach is tested on spare-parts and maintenance workflows in OpenMES instead of generic websites.
- **Independent oracle:** the verdict comes from rule-based checks, not from the LLM's own judgement.
- **Controlled comparison:** the agent is compared with a Playwright suite on scenarios with known, injected defects.
- **Complete reporting:** cost, execution time and human intervention are reported together with detection accuracy.

The next step is to implement the agent and the verification engine described in Section 3.2 and to run the benchmark. Its results will show how far autonomous agents can support end-to-end quality assurance of Manufacturing Execution Systems in smart factories.

References

[1] B. Saenz de Ugarte, A. Artiba, and R. Pellerin, “Manufacturing execution system – a literature review,” *Production Planning & Control*, vol. 20, no. 6, pp. 525–539, 2009.

[2] International Electrotechnical Commission, *IEC 62264-1:2013 – Enterprise-control system integration – Part 1: Models and terminology*. Geneva, Switzerland: IEC, 2013.

[3] Y. Lu, “Industry 4.0: A survey on technologies, applications and open research issues,” *Journal of Industrial Information Integration*, vol. 6, pp. 1–10, 2017.

[4] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.

[5] M. Hammoudi, G. Rothermel, and P. Tonella, “Why do record/replay tests of web applications break?” in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2016.

[6] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella, “Approaches and tools for automated end-to-end web testing,” *Advances in Computers*, vol. 101, pp. 193–237, 2016.

[7] A. Stocco, R. Yandrapally, and A. Mesbah, “Visual web test repair,” in *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018.

[8] Microsoft, “Playwright: Fast and reliable end-to-end testing for modern web apps,” documentation.

[Online]. Available: https://playwright.dev/

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[11] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, art. 186345, 2024.

[12] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[13] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[14] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR vol. 235, 2024.

[15] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[16] Anthropic, "Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku," Oct. 2024. [Online]. Available: https://www.anthropic.com/news/3-5-models-and-computer-use

[17] OpenAI, "Computer-using agent," Jan. 2025. [Online]. Available: https://openai.com/index/computer-using-agent/

[18] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[19] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing

expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[20] J. Yoon, R. Feldt, and S. Yoo, "Intent-driven mobile GUI testing with autonomous large language model agents," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2024.

[21] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[22] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[23] OpenMES Contributors, "OpenMES: Open-source manufacturing execution system," GitHub repository. [Online]. Available: https://github.com/Mes-Open/OpenMes